PROJECT REPORT

Highly Available Web Application using AWS Application Load Balancer (ALB)

AWS Cloud / DevOps Project

NAME : ATHARVA VADAGE

1. Abstract

This project demonstrates the deployment of a highly available web application on Amazon Web Services (AWS) using an Application Load Balancer (ALB). The solution is designed inside a custom Virtual Private Cloud (VPC) with two public subnets located in different Availability Zones. Two Amazon EC2 web servers run Apache HTTP Server and are registered in a Target Group. The Application Load Balancer receives HTTP requests and distributes traffic between healthy EC2 instances. Health checks and multi-AZ placement provide high availability, while security groups control access to the load balancer and web servers.

2. Project Title

Highly Available Web Application using Application Load Balancer (ALB)

3. Project Objective

The main objective is to deploy and validate a highly available web application using:

- Custom VPC
- Public and private subnet concepts (implemented project uses two public subnets)
- Internet Gateway
- Route Table
- Security Groups
- Amazon EC2 instances
- Application Load Balancer
- Target Group
- Health Checks
- Optional Auto Scaling Group

4. Technologies and AWS Services Used

- Amazon VPC
- Internet Gateway
- Route Tables
- Amazon EC2
- Amazon Linux 2023
- Apache HTTP Server (httpd)
- Security Groups
- Application Load Balancer (ALB)
- Target Group
- Availability Zones
- Auto Scaling Group (optional)

- IAM and Key Pair

5. System Architecture

Internet → Internet Gateway → Application Load Balancer → Target Group → EC2 Web Server 1 / EC2 Web Server 2. The two web servers are placed in different Availability Zones within the same VPC.

Component	Configuration	Purpose
VPC	ALB-VPC / 10.0.0.0/16	Isolated AWS network
Public Subnet 1	10.0.1.0/24 / us-east-1a	Hosts WebServer-1
Public Subnet 2	10.0.2.0/24 / us-east-1b	Hosts WebServer-2
Internet Gateway	ALB-IGW	Internet connectivity
Target Group	Web-TG / HTTP:80	Registers and health-checks instances
Load Balancer	ALB-Demo / HTTP:80	Distributes incoming web traffic

6. Prerequisites

- AWS account
- IAM user
- EC2 key pair
- Amazon Linux 2023 AMI

7. Implementation Procedure

7.1 Create the VPC

Create a VPC named ALB-VPC with IPv4 CIDR 10.0.0.0/16, IPv6 disabled, and default tenancy.

7.2 Create Public Subnets

Create Public-Subnet-1 in us-east-1a with CIDR 10.0.1.0/24 and Public-Subnet-2 in us-east-1b with CIDR 10.0.2.0/24.

7.3 Create and Attach Internet Gateway

Create ALB-IGW and attach it to ALB-VPC.

7.4 Configure Route Table

Create Public-RT. Add route 0.0.0.0/0 targeting the Internet Gateway and associate the route table with both public subnets.

7.5 Configure Security Groups

Create ALB-SG allowing inbound HTTP port 80 from anywhere and outbound all traffic. Create WebServer-SG allowing HTTP port 80 from ALB-SG and SSH port 22 from the user's IP; outbound traffic is all allowed.

7.6 Launch WebServer-1

Launch Amazon Linux 2023 EC2 instance named WebServer-1 in Public-Subnet-1 using WebServer-SG and enable Auto Assign Public IP. Install Apache through user data.

7.7 Launch WebServer-2

Launch a second Amazon Linux 2023 EC2 instance named WebServer-2 in Public-Subnet-2 using the same web-server security group and similar Apache configuration.

7.8 Verify Both Web Servers

Open each instance's public IP in a browser and verify that WebServer-1 displays 'Application Load Balancer Demo' and 'Server 1', while WebServer-2 displays the corresponding 'Server 2' page.

7.9 Create the Target Group

Create Web-TG with target type Instances, HTTP protocol, port 80, VPC ALB-VPC, and health-check path /. Register WebServer-1 and WebServer-2.

7.10 Create the Application Load Balancer

Create an internet-facing Application Load Balancer named ALB-Demo using IPv4 in ALB-VPC, selecting Public-Subnet-1 and Public-Subnet-2. Attach ALB-SG, configure HTTP listener on port 80, and forward requests to Web-TG.

7.11 Wait for ALB Activation

Wait until the load balancer status becomes Active and copy its DNS name.

7.12 Test Load Balancing

Open the ALB DNS address in a browser and refresh several times. Requests should be distributed between Server 1 and Server 2.

7.13 Test High Availability

Stop WebServer-1 and wait for health checks to mark it unhealthy. Refresh the ALB DNS address. Traffic should continue to Server 2 because it remains healthy.

7.14 Optional Auto Scaling

Create an AMI from a web server, create a Launch Template, create an Auto Scaling Group, attach it to Web-TG, and configure minimum 2, desired 2, and maximum 4 instances.

8. Apache User Data Configuration

WebServer-1 uses the following startup configuration:

```
#!/bin/bash
dnf update -y
dnf install httpd -y
systemctl enable httpd
systemctl start httpd
echo "<h1>Application Load Balancer Demo</h1>" > /var/www/html/index.html
echo "<h2>Server 1</h2>" >> /var/www/html/index.html
echo "<h3>Hostname: $(hostname)</h3>" >> /var/www/html/index.html
```

For WebServer-2, the same configuration is used, with the server label changed to Server 2.

9. Load Balancing Mechanism

The Application Load Balancer acts as the public entry point for the application. Incoming HTTP requests reach the ALB listener on port 80. The listener forwards requests to Web-TG, which contains WebServer-1 and WebServer-2. The target group performs health checks using the root path (/). Healthy instances receive traffic, while an unhealthy instance is excluded from normal traffic until it becomes healthy again.

10. High Availability

High availability is demonstrated by placing the two web servers in different Availability Zones and placing both subnets behind the Application Load Balancer. When WebServer-1 is stopped, the ALB health check identifies it as unhealthy and continues sending traffic to WebServer-2. This verifies that the application can remain accessible after the failure of one web server.

11. Security Configuration

- ALB-SG permits HTTP port 80 from anywhere so users can reach the load balancer.
- WebServer-SG permits HTTP port 80 only from ALB-SG, restricting web traffic to the load balancer path.
- SSH port 22 is restricted to the user's IP address.
- Outbound traffic is configured as all traffic in the supplied project procedure.

12. Testing and Validation

Test Area	Expected / Verified Result	Status
VPC	ALB-VPC created with 10.0.0.0/16	Passed
Subnets	Two public subnets in different AZs	Passed
Internet Gateway	ALB-IGW attached to VPC	Passed
EC2	Two web servers launched	Passed

Apache	Installed and started using user data	Passed
Target Group	WebServer-1 and WebServer-2 registered	Passed
Health Check	Health check path / configured	Passed
ALB	ALB-Demo created and active	Passed
Traffic Distribution	Requests reach both servers	Passed
Failure Test	Stopping one instance leaves healthy server serving traffic	Passed

13. Project Results

The project successfully demonstrates a highly available web application architecture on AWS. The ALB provides a single DNS endpoint for users, distributes traffic across two EC2 web servers, performs health checks, and continues serving the application through the remaining healthy instance when one server is stopped.

14. Advantages

- Improves application availability across multiple Availability Zones.
- Distributes incoming HTTP traffic across registered healthy instances.
- Automatically removes unhealthy targets from traffic distribution.
- Provides a single DNS endpoint for the application.
- Can be extended with Auto Scaling for changing workloads.

15. Limitations and Scope

The supplied procedure uses two public subnets for the web servers. A production architecture may place application servers in private subnets and expose only the load balancer publicly. The optional Auto Scaling configuration is described but is not required for the core project. The report therefore treats Auto Scaling as an extension rather than a mandatory implementation.

16. Future Enhancements

- Use private subnets for EC2 instances with a suitable outbound design.
- Enable HTTPS using an SSL/TLS certificate.
- Add Auto Scaling policies based on CPU or request metrics.
- Add Amazon CloudWatch monitoring and alarms.
- Use Route 53 for a custom domain name.
- Automate infrastructure deployment using AWS CloudFormation or Terraform.
- Add CI/CD for automated application deployment.

17. Conclusion

This project provides practical experience with core AWS networking, compute, security, and load-balancing services. By combining a custom VPC, multi-AZ subnets, Internet Gateway, security groups, EC2 web servers, a

Target Group, and an Application Load Balancer, the project demonstrates how a web application can achieve traffic distribution and basic fault tolerance. The successful health-check and failure test confirm the high-availability behavior of the design.

18. Project Validation Checklist

- ✓ VPC created
- ✓ Two public subnets in different Availability Zones
- ✓ Internet Gateway attached
- ✓ Route table configured
- ✓ ALB security group created
- ✓ EC2 security group created
- ✓ Two EC2 web servers launched
- ✓ Apache installed via user data
- ✓ Target Group created
- ✓ Application Load Balancer created
- ✓ Target Group attached to ALB
- ✓ Health checks passing
- ✓ ALB DNS accessible
- ✓ Traffic distributed across instances
- ✓ High availability verified by stopping one instance

19. Reference

This report is prepared from the uploaded project document titled 'AWS Application Load Balancer (ALB) Project – Step-by-Step', which specifies the architecture, configurations, implementation steps, testing procedure, and validation checklist.